

InternNova Data Analytics Internship

Week 4 Assignment: Statistics, Data Visualization & Exploratory Data Analysis

Submitted by: Brojo Mohan Dutta

Duration: 1 Week (7 Days) | Total Marks: 100

Python source code for all tasks

Screenshots of program outputs

Brief explanation of each task

Dataset Used: sales_data.csv

sales_data.csv

order_id	order_date	product_category	region	units_sold	unit_price	customer_age	customer_rating	payment_method	revenue
1001	2024-01-01	Books	West	9	774.05	29	4.7	UPI	6966.45
1002	2024-01-04	Sports	East	12	3623.57	56	1.6	Cash on Delivery	43482.84
1003	2024-01-07	Home & Kitchen	West	12	3851.77	19	2.4	Debit Card	46221.24
1004	2024-01-10	Sports	East	12	2894.13	20	1.5	Debit Card	34729.56
1005	2024-01-13	Sports	West	13	3900.64	54	4.7	UPI	50708.32
1006	2024-01-16	Clothing	North	9	2570.22	34		Credit Card	23131.98
1007	2024-01-19	Home & Kitchen	West	13	2709.12	19	2.0	UPI	35218.56
1008	2024-01-22	Home & Kitchen	East	13	2252.2	19	3.6	Credit Card	29278.6
1009	2024-01-25	electronics	East	1	322.01	45	4.3	Cash on Delivery	322.01
1010	2024-01-28	Sports	South	9	717.88	40	3.2	Debit Card	6460.92
1011	2024-01-31	Books	North	7	350.86	54	3.1	Cash on Delivery	2456.02
1012	2024-02-03	Home & Kitchen	West	9	3254.77	49	2.0	Credit Card	29292.93
1013	2024-02-06	Sports	South	8		50	1.4	Credit Card	
1014	2024-02-09	Clothing	West	1	2641.14	18	4.6	Cash on Delivery	2641.14
1015	2024-02-12	Books	West	12	4556.32	36	4.6	Credit Card	54675.84
1016	2024-02-15	Clothing	South	8	1396.6	19	3.5	Cash on Delivery	11172.8
1017	2024-02-18	Books	South	8	2169.84	43	2.4	Debit Card	17358.72
1018	2024-02-21	Sports	South	11	3826.65	49	2.4	Cash on Delivery	42093.15
1019	2024-02-24	Electronics	South	3	1298.23	23	3.9	Debit Card	3894.69
1020	2024-02-27	SPORTS	South	1	569.5	49	4.6	Cash on Delivery	569.5
1021	2024-03-01	Clothing	West	8	1590.81	21	4.5	Credit Card	12726.48
1022	2024-03-04	Sports	South	3	973.86	28	4.1	Cash on Delivery	2921.58
1023	2024-03-07	Books	North	3	4662.55	34	3.6	Credit Card	13987.65
1024	2024-03-10	Electronics	East	1	4078.98	55	1.3	Cash on Delivery	4078.98
1025	2024-03-13	Electronics	South	11	3240.34	41	1.6	Cash on Delivery	35643.74
1026	2024-03-16	Home & Kitchen	South	5	4383.01	22	4.6	UPI	21915.05
1027	2024-03-19	Home & Kitchen	West	10	4057.63	51	3.4	Credit Card	40576.3
1028	2024-03-22	Clothing	South	7	1095.54	23		UPI	7668.78
1029	2024-03-25	Books	South	10	4484.28	39	1.4	Credit Card	44842.8
1030	2024-03-28	Books	South	9	2788.84	28	3.7	UPI	25099.56

1031	2024-03-31	Home & Kitchen	West	12	4075.71	33	1.0	Debit Card	48908.52
1032	2024-04-03	Books	South	7	4501.24	50	1.6	Cash on Delivery	31508.68
1033	2024-04-06	Books	East	9	1726.42	26	3.2	Credit Card	15537.78
1034	2024-04-09	Electronics	West	8		23	3.8	Credit Card	
1035	2024-04-12	Home & Kitchen	East	12	1294.09	33	3.6	Cash on Delivery	15529.08
1036	2024-04-15	Sports	West	2	2250.12	46	1.9	Credit Card	4500.24
1037	2024-04-18	Home & Kitchen	South	1	4126.47	20	3.8	Credit Card	4126.47
1038	2024-04-21	Sports	East	7	4331.51	37	1.9	UPI	30320.57
1039	2024-04-24	Electronics	West	7	233.37	53	2.3	Credit Card	1633.59
1040	2024-04-27	Clothing	North	14	2651.59	36	4.0	Debit Card	37122.26
1041	2024-04-30	Books	South	120	2203.57	43	3.6	Debit Card	264428.4
1042	2024-05-03	Electronics	West	5	1266.12	20	4.4	Cash on Delivery	6330.6
1043	2024-05-06	Books	North	3	775.35	36	3.6	Credit Card	2326.05
1044	2024-05-09	Clothing	West	12	1820.55	37	3.3	Cash on Delivery	21846.6
1045	2024-05-12	Clothing	North	8	4725.97	49	1.4	Debit Card	37807.76
1046	2024-05-15	Electronics	South	6	1751.37	24	2.5	Credit Card	10508.22
1047	2024-05-18	Clothing	East	11	2690.19	58	2.1	Credit Card	29592.09
1048	2024-05-21	Sports	North	3	3574.49	50	2.0	Cash on Delivery	10723.47
1049	2024-05-24	Clothing	West	1	1945.42	57	4.9	Debit Card	1945.42
1050	2024-05-27	Books	South	3	4864.55	56	2.6	Credit Card	14593.65
1004	2024-01-10	Sports	East	12	2894.13	20	1.5	Debit Card	34729.56
1016	2024-02-15	Clothing	South	8	1396.6	19	3.5	Cash on Delivery	11172.8

Objective

This project applies **statistics, data visualization, and Exploratory Data Analysis (EDA)** to a retail sales dataset. It covers descriptive statistics (mean, median, mode, variance, std dev), correlation and probability, outlier detection via IQR, visual analysis with Matplotlib and Seaborn, a full data cleaning workflow (missing values, duplicates, inconsistent categories), and closes with data-driven business recommendations. Cell 1: Setup & Load Data

Explanation: Imports the libraries used throughout the notebook and loads the dataset into a DataFrame.

Python Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("sales_data.csv")
df.head()
```

Output Screenshot:

	order_id	order_date	product_category	region	units_sold	unit_price	customer_age	customer_rating	payment_method	revenue
0	1001	2024-01-01	Books	West	9	774.05	29	4.7	UPI	6966.45
1	1002	2024-01-04	Sports	East	12	3623.57	56	1.6	Cash on Delivery	43482.84
2	1003	2024-01-07	Home & Kitchen	West	12	3851.77	19	2.4	Debit Card	46221.24
3	1004	2024-01-10	Sports	East	12	2894.13	20	1.5	Debit Card	34729.56
4	1005	2024-01-13	Sports	West	13	3900.64	54	4.7	UPI	50708.32

Task 1: Mean, Median & Mode

Explanation: Calculates the three central-tendency measures on `revenue` (mean/median) and `units_sold` (mode). Mean shows the average order value but is pulled upward by the outlier order; median is more robust to that outlier and better represents a "typical" order; mode shows the most commonly ordered quantity.

Python Code:

```
mean_revenue = df["revenue"].mean()
median_revenue = df["revenue"].median()
mode_units_sold = df["units_sold"].mode()[0]

print("Mean Revenue:", round(mean_revenue, 2))
print("Median Revenue:", round(median_revenue, 2))
print("Mode of Units Sold:", mode_units_sold)
```

Output Screenshot:

```
Mean Revenue: 25706.56
Median Revenue: 16448.25
Mode of Units Sold: 12
```

Explanation of results: Mean revenue (₹25,706.56) is much higher than the median (₹16,448.25) — a clear sign of a right-skewed distribution pulled by the outlier order (120 units). Mode of `units_sold` (12) shows that ordering 12 units is the most frequent order size.

Task 2: Variance & Standard Deviation

Explanation: Computes variance and standard deviation of `revenue` to quantify how spread out order values are around the mean.

Python Code:

```
variance_revenue = df["revenue"].var()
std_revenue = df["revenue"].std()

print("Variance of Revenue:", round(variance_revenue, 2))
print("Standard Deviation of Revenue:", round(std_revenue, 2))
```

Output Screenshot:

```
Variance of Revenue: 1444248098.37
Standard Deviation of Revenue: 38003.26
```

Explanation of results: A standard deviation of ~₹38,003 (larger than the median itself) indicates high variability in order values — driven mainly by the single 120-unit outlier order, confirming that revenue is not tightly clustered around the average.

Task 3: Correlation & Probability Basics

Explanation: Correlation checks the linear relationship between `unit_price` and `revenue`, and separately between `customer_age` and `customer_rating`. The probability example calculates the real-world chance a random order was paid via Credit Card..

Python Code:

```
# Correlation
corr_price_revenue = df["unit_price"].corr(df["revenue"])
corr_age_rating = df["customer_age"].corr(df["customer_rating"])

print("Correlation (unit_price vs revenue):", round(corr_price_revenue, 3))
print("Correlation (customer_age vs customer_rating):", round(corr_age_rating, 3))

# Probability example: chance an order was paid by Credit Card
total_orders = len(df)
credit_card_orders = len(df[df["payment_method"] == "Credit Card"])
probability_credit_card = credit_card_orders / total_orders

print(f"P(Payment Method = Credit Card) = {credit_card_orders}/{total_orders} = {round(probability_credit_card, 3)}")
```

Output Screenshot:

```
Correlation (unit_price vs revenue): 0.245
Correlation (customer_age vs customer_rating): -0.279
P(Payment Method = Credit Card) = 18/52 = 0.346
```

Explanation of results: `unit_price` vs `revenue` shows a **weak positive** correlation (0.245) — expected, since revenue also depends on `units_sold`, not price alone. `customer_age` vs `customer_rating` shows a **weak negative** correlation (-0.279), suggesting a slight tendency for older customers to give marginally lower ratings, though the relationship is not strong. Probability of a Credit Card payment is **34.6%** (18 of 52 orders).

Task 4: Outlier Detection

Explanation: Uses the IQR (Interquartile Range) method on `units_sold` — values beyond $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ are flagged as outliers.

Python Code:

```
Q1 = df["units_sold"].quantile(0.25)
Q3 = df["units_sold"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["units_sold"] < lower_bound) | (df["units_sold"] > upper_bound)]

print("Q1:", Q1, "| Q3:", Q3, "| IQR:", IQR)
print("Lower Bound:", lower_bound, "| Upper Bound:", upper_bound)
print("Outliers detected:\n", outliers[["order_id", "units_sold", "revenue"]])
```

Output Screenshot:

```
Q1: 4.5 | Q3: 11.25 | IQR: 6.75
Lower Bound: -5.625 | Upper Bound: 21.375
Outliers detected:
   order_id  units_sold  revenue
40      1041          120  264428.4
```

Explanation of results: With $Q1=4.5$, $Q3=11.25$, and an upper bound of 21.375, order **1041** (`units_sold = 120`) is flagged as a clear outlier — over 5x the upper bound. Outliers like this can distort the mean, inflate variance/std deviation, and mislead trend analysis if not handled — they should be investigated (data entry error vs. genuine bulk order) before deciding to cap, remove, or keep them.

Cell 2: Prep for Visualization

Explanation: Converts `order_date` to a proper datetime type so the line chart plots chronologically.

Python Code:

```
df["order_date"] = pd.to_datetime(df["order_date"])
# Group total revenue by order date
daily_revenue = (df.groupby("order_date")["revenue"].sum().reset_index())
# Sort by date
daily_revenue = daily_revenue.sort_values("order_date")
```

Task 5: Matplotlib Visualization

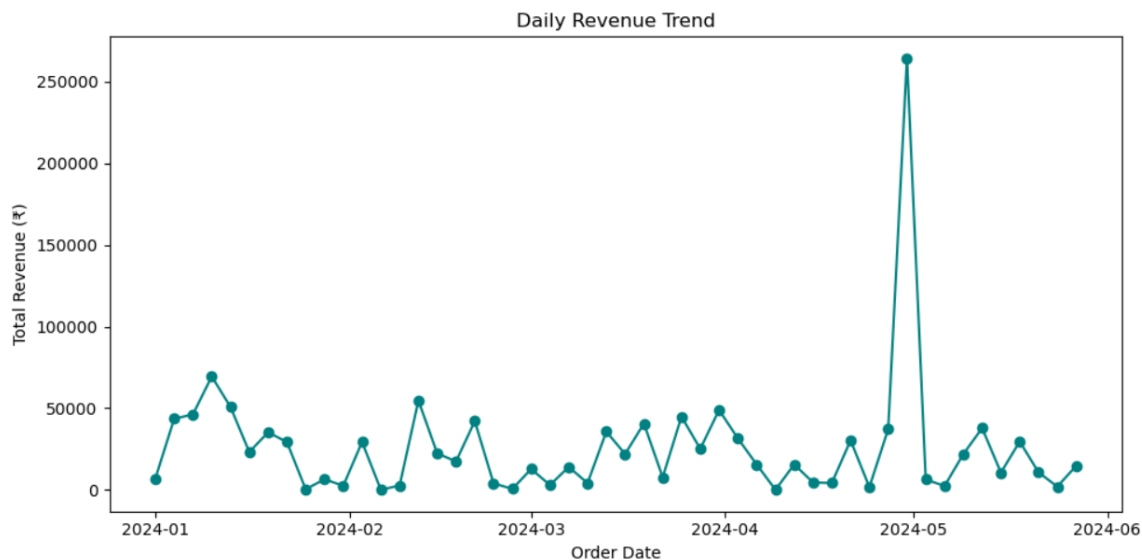
Explanation: Five Matplotlib charts, each on a different variable pair, to explore the sales data from different angles.

1. Line Chart — Revenue trend over time

Python Code:

```
plt.figure(figsize=(10, 5))
plt.plot(daily_revenue["order_date"],daily_revenue["revenue"],marker="o", color="teal")
plt.title("Daily Revenue Trend")
plt.xlabel("Order Date")
plt.ylabel("Total Revenue (₹)")
plt.tight_layout()
plt.show()
```

Output Screenshot:



Explanation: Tracks order-by-order revenue chronologically. A single sharp spike stands out around late April — the 120-unit outlier order — otherwise revenue fluctuates in a normal ₹1,000–₹55,000 band.

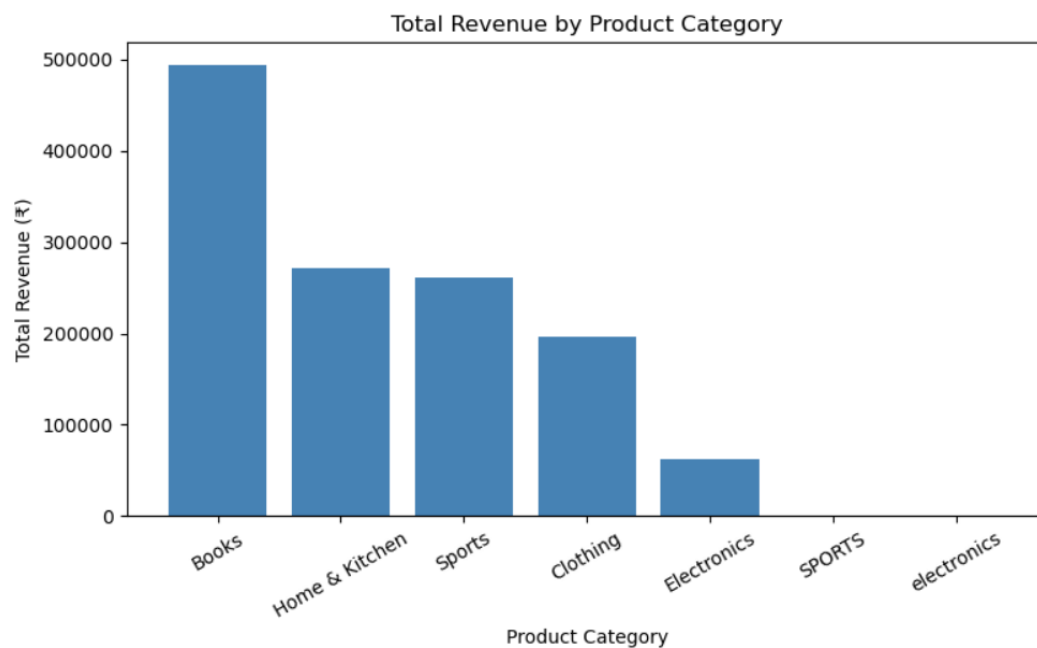
2. Bar Chart — Total revenue by category

Python Code:

```
cat_revenue = df.groupby("product_category")["revenue"].sum().sort_values(ascending=False)

plt.figure(figsize=(8,5))
plt.bar(cat_revenue.index, cat_revenue.values, color="steelblue")
plt.title("Total Revenue by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Revenue (₹)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Explanation: Books leads total revenue (₹493,782) — largely inflated by the outlier order — followed by Home & Kitchen and Sports. Note "electronics" and "SPORTS" appear as separate tiny bars here because casing hasn't been cleaned yet — this issue gets fixed in Task 7.

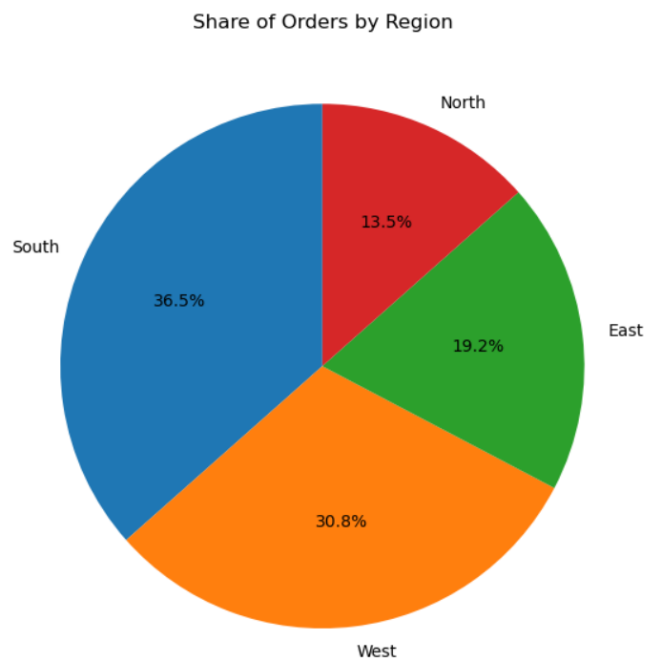
3. Pie Chart — Share of orders by region

Python Code:

```
region_orders = df["region"].value_counts()

plt.figure(figsize=(6,6))
plt.pie(region_orders.values, labels=region_orders.index, autopct="%1.1f%%", startangle=90)
plt.title("Share of Orders by Region")
plt.tight_layout()
plt.show()
```

Output Screenshot:



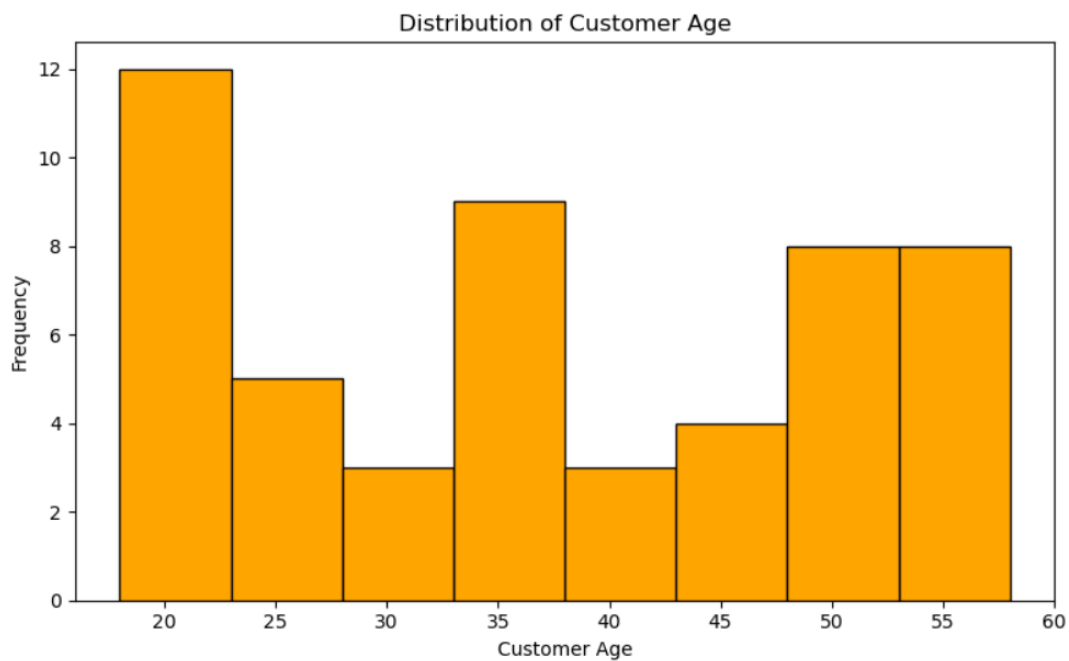
Explanation: **South** dominates order share (~36.5%), followed by **West** (~30.8%); **North** has the smallest share (~13.5%) — useful for spotting regional demand concentration..

4. Histogram — Customer age distribution

Python Code:

```
plt.figure(figsize=(8,5))
plt.hist(df["customer_age"], bins=8, color="orange", edgecolor="black")
plt.title("Distribution of Customer Age")
plt.xlabel("Customer Age")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

Output Screenshot:



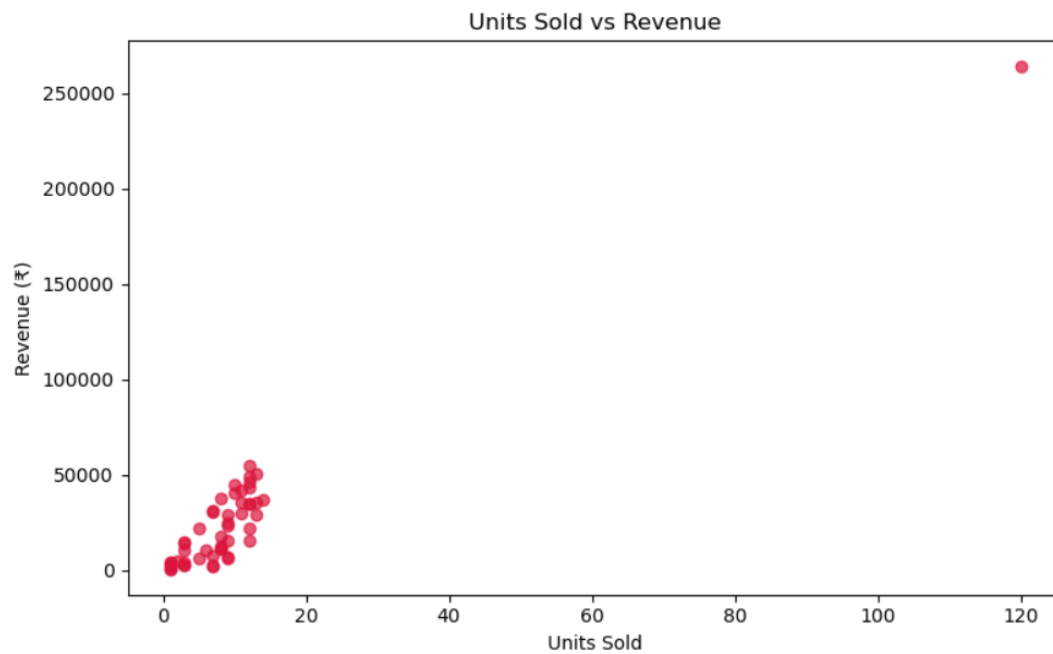
Explanation: Shows the frequency of customers across age bands, revealing whether the customer base skews toward a particular generation — helpful for targeted marketing.

5. Scatter Plot — Units sold vs revenue

Python Code:

```
plt.figure(figsize=(8,5))
plt.scatter(df["units_sold"], df["revenue"], color="crimson", alpha=0.7)
plt.title("Units Sold vs Revenue")
plt.xlabel("Units Sold")
plt.ylabel("Revenue (₹)")
plt.tight_layout()
plt.show()
```

Output Screenshot:



Explanation: Strong positive relationship between units sold and revenue (confirmed numerically in Task 8, $\text{corr} \approx 0.96$) — expected, since revenue is directly derived from $\text{units} \times \text{price}$. The outlier order (120 units) sits far to the right, isolated from the main cluster.

Task 6: Seaborn Visualization

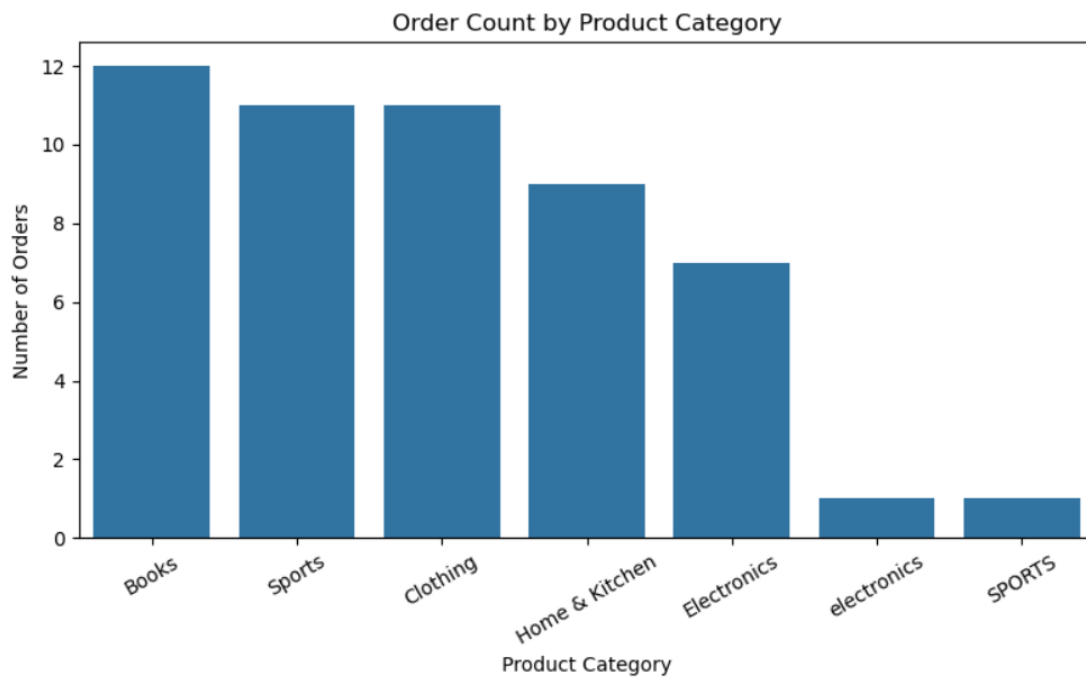
Explanation: Four Seaborn plots that add statistical and relational depth beyond what Matplotlib easily shows.

1. Count Plot — Orders per category

Python Code:

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x="product_category",
order=df["product_category"].value_counts().index)
plt.title("Order Count by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Number of Orders")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Output Screenshot:



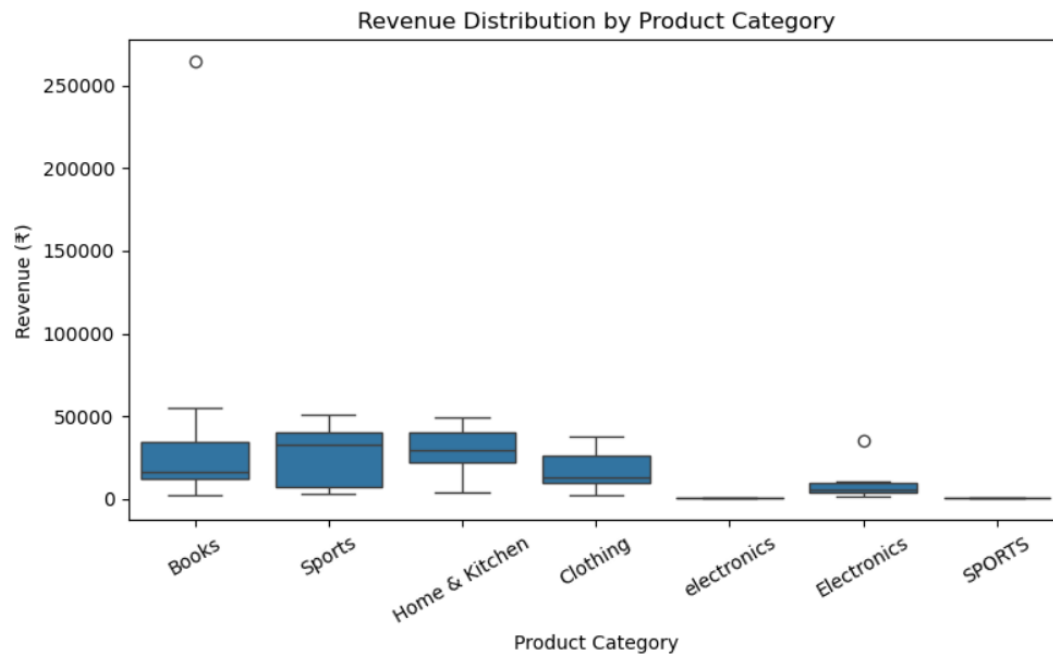
Explanation: Counts orders per category label as-is (pre-cleaning), so `Sports`, `SPORTS`, and `electronics`/`Electronics` show as separate bars — visually flags the inconsistent-text problem tackled in Task 7.

2. Box Plot — Revenue spread by category

Python Code:

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="product_category", y="revenue")
plt.title("Revenue Distribution by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Revenue (₹)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Output Screenshot:



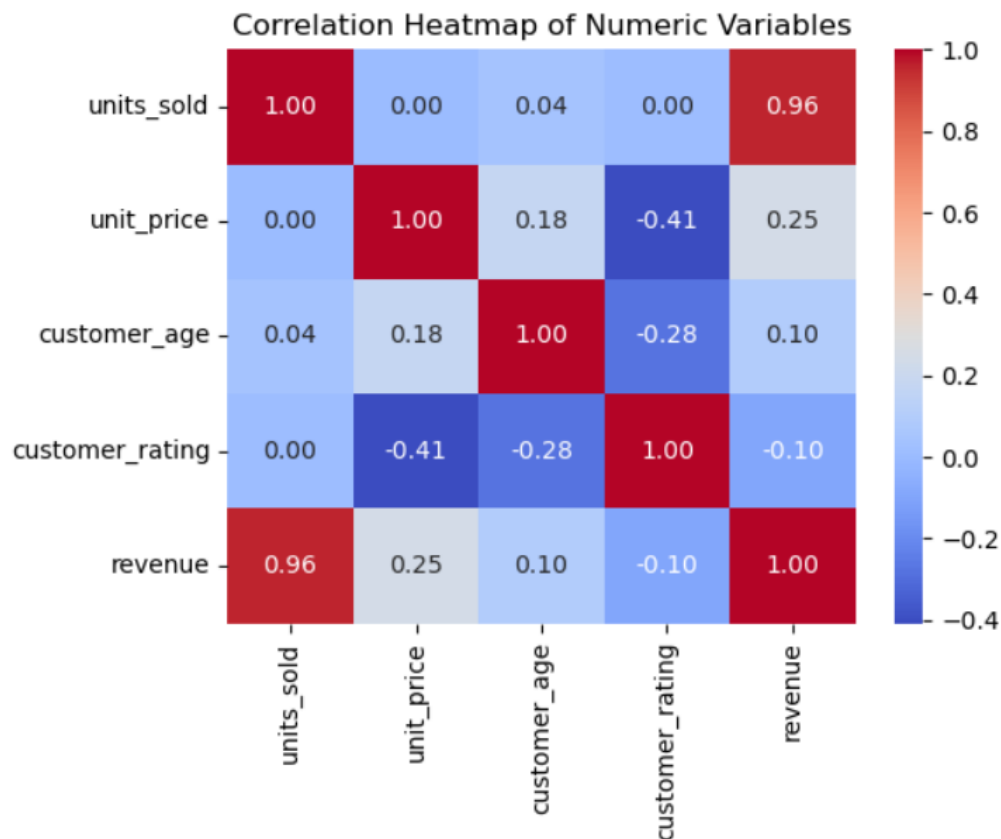
Explanation: Shows the median, IQR, and spread of revenue per category. **Books** shows a clear high outlier point far above its box — the same 120-unit order flagged earlier — visually confirming the Task 4 outlier detection.

3. Heatmap — Correlation between numeric variables

Python Code:

```
numeric_df = df[["units_sold", "unit_price", "customer_age", "customer_rating", "revenue"]]  
  
plt.figure(figsize=(6,5))  
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")  
plt.title("Correlation Heatmap of Numeric Variables")  
plt.tight_layout()  
plt.show()
```

Output Screenshot:



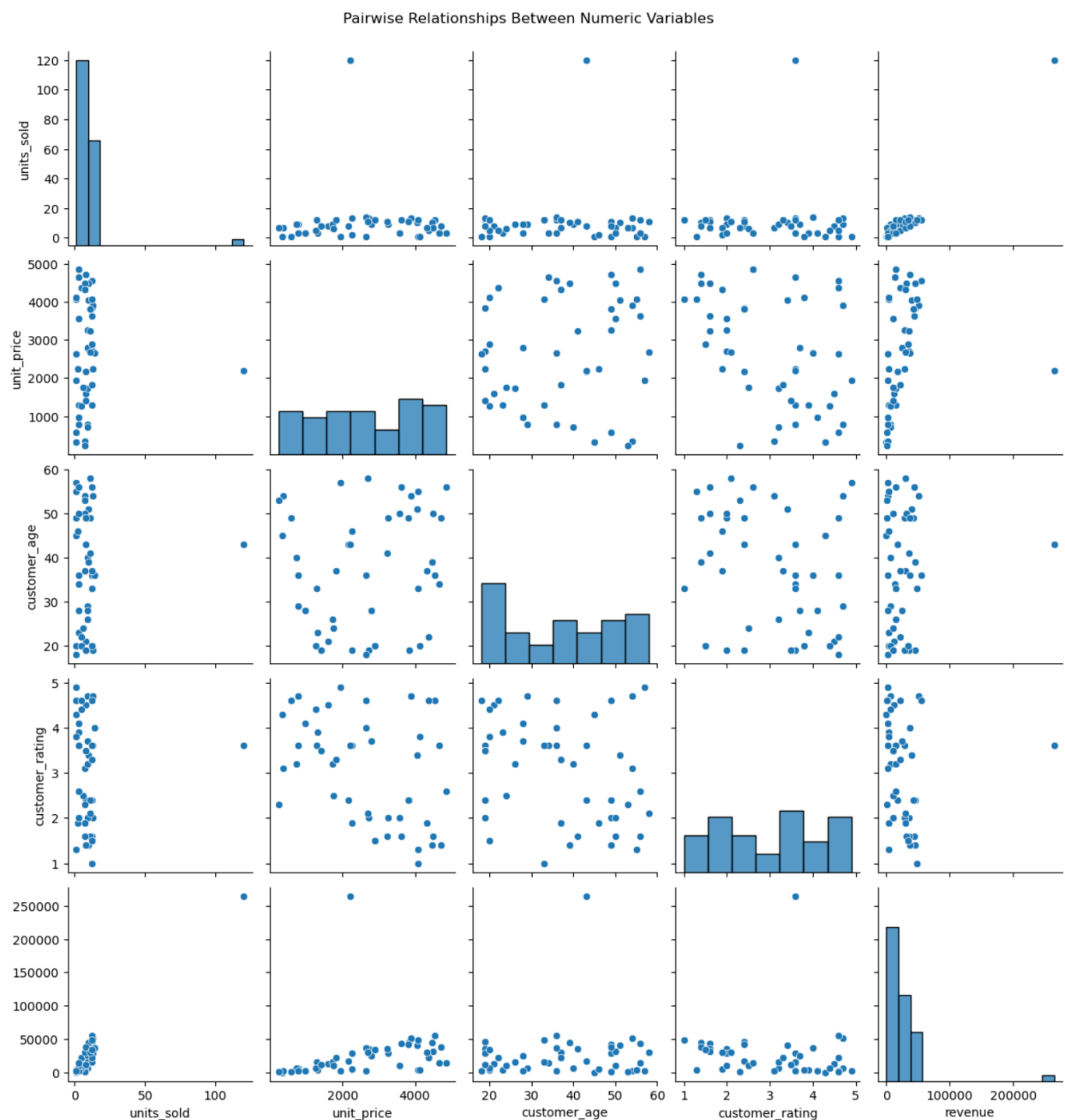
Explanation: units_sold and revenue show the strongest correlation (**0.96**), as expected. unit_price and customer_rating show a moderate negative correlation (**-0.41**) — pricier items trending toward slightly lower ratings in this sample. All other pairs are weakly correlated..

4. Pair Plot — Pairwise relationships

Python Code:

```
g = sns.pairplot(numeric_df.dropna())
g.fig.suptitle("Pairwise Relationships Between Numeric Variables", y=1.02)
plt.show()
```

Output Screenshot:



Explanation: Lays out every numeric variable against every other, with distributions on the diagonal. The units_sold vs revenue panel clearly shows the tight positive linear relationship (and the one outlier point breaking away from the cluster), while other variable pairs show scattered, weak relationships.

Task 7: EDA – Data Inspection & Cleaning

Explanation: Full inspection pass followed by a cleaning pass — fixing inconsistent category text, imputing missing values, and dropping duplicates.

Data Inspection

Python Code:

```
print("Shape (rows, columns):", df.shape)
print("Column Names:", df.columns.tolist())
print("Data Types:\n", df.dtypes)
print("Statistical Summary:\n", df.describe())
```

Output Screenshot:

```
Shape (rows, columns): (52, 10)
Column Names: ['order_id', 'order_date', 'product_category', 'region', 'units_sold', 'unit_price', 'customer_age', 'customer_rating', 'payment_method', 'revenue']
Data Types:
order_id          int64
order_date        datetime64[ns]
product_category  object
region            object
units_sold        int64
unit_price        float64
customer_age      int64
customer_rating   float64
payment_method    object
revenue           float64
dtype: object
Statistical Summary:

```

	order_id	order_date	units_sold	unit_price
count	52.000000	52	52.000000	50.000000
mean	1024.903846	2024-03-12 17:04:36.923076864	9.788462	2602.683400
min	1001.000000	2024-01-01 00:00:00	1.000000	233.370000
25%	1012.750000	2024-02-05 06:00:00	4.500000	1396.600000
50%	1024.500000	2024-03-11 12:00:00	8.000000	2646.365000
75%	1037.250000	2024-04-18 18:00:00	11.250000	3888.422500
max	1050.000000	2024-05-27 00:00:00	120.000000	4864.550000
std	14.650516	NaN	16.066451	1392.463733

	customer_age	customer_rating	revenue
count	52.000000	50.000000	50.000000
mean	36.403846	3.024000	25706.560000
min	18.000000	1.000000	322.010000
25%	23.000000	2.000000	6363.180000
50%	36.000000	3.250000	16448.250000
75%	49.000000	3.875000	35096.310000
max	58.000000	4.900000	264428.400000
std	13.286082	1.161378	38003.264312

Before-cleaning output (key figures): Shape (52, 10). unit_price, customer_rating, revenue each have 2 missing values (48/50 non-null out of 52).

Data Cleaning

Python Code:

```
# 1. Identify missing values
print("Missing values before cleaning:\n", df.isnull().sum())

# 2. Check duplicates
print("Duplicate rows:", df.duplicated().sum())

# 3. Check inconsistent category text
print("Unique product_category values (raw):", df["product_category"].unique())

# --- Clean ---
df_clean = df.copy()

# Fix inconsistent text (electronics/SPORTS -> Title Case)
```



```

df_clean["product_category"] = df_clean["product_category"].str.strip().str.title()

# Handle missing values: median imputation for price/rating, recompute revenue
df_clean["customer_rating"] =
df_clean["customer_rating"].fillna(df_clean["customer_rating"].median())
df_clean["unit_price"] = df_clean["unit_price"].fillna(df_clean["unit_price"].median())
df_clean["revenue"] = df_clean["units_sold"] * df_clean["unit_price"]

# Remove duplicate rows
df_clean = df_clean.drop_duplicates()

print("Unique product_category values (cleaned):", df_clean["product_category"].unique())
print("Missing values after cleaning:\n", df_clean.isnull().sum())
print("Duplicate rows after cleaning:", df_clean.duplicated().sum())
print("Shape after cleaning:", df_clean.shape)

df_clean.to_csv("sales_data_cleaned.csv", index=False)

```

Output Screenshot:

```

Missing values before cleaning:
  order_id      0
  order_date    0
  product_category  0
  region        0
  units_sold    0
  unit_price    2
  customer_age  0
  customer_rating  2
  payment_method  0
  revenue       2
dtype: int64
Duplicate rows: 2
Unique product_category values (raw): ['Books' 'Sports' 'Home & Kitchen' 'Clothing' 'electronics' 'Electronics'
'SPORTS']
Unique product_category values (cleaned): ['Books' 'Sports' 'Home & Kitchen' 'Clothing' 'Electronics']
Missing values after cleaning:
  order_id      0
  order_date    0
  product_category  0
  region        0
  units_sold    0
  unit_price    0
  customer_age  0
  customer_rating  0
  payment_method  0
  revenue       0
dtype: int64
Duplicate rows after cleaning: 0
Shape after cleaning: (50, 10)

```

Before vs After:

Check	Before	After
Shape	(52, 10)	(50, 10)
Missing values	2 each in unit_price, customer_rating, revenue	0
Duplicate rows	2	0
Unique category labels	7 (Sports, SPORTS, Electronics, electronics, ...)	5 (clean)

Task 8: EDA – Correlation & Insights

Explanation: Re-runs correlation on the cleaned data and aggregates revenue/rating by category, region, and payment method to surface patterns.

Python Code:

```
numeric_df = df_clean[["units_sold", "unit_price", "customer_age", "customer_rating",
"revenue"]]
print("Correlation matrix:\n", numeric_df.corr())

revenue_by_category =
df_clean.groupby("product_category")["revenue"].sum().sort_values(ascending=False)
print("Revenue by category:\n", revenue_by_category)

revenue_by_region =
df_clean.groupby("region")["revenue"].sum().sort_values(ascending=False)
print("Revenue by region:\n", revenue_by_region)

payment_share = df_clean["payment_method"].value_counts(normalize=True) * 100
print("Payment method share (%):\n", payment_share)

rating_by_category =
df_clean.groupby("product_category")["customer_rating"].mean().sort_values()
print("Average rating by category:\n", rating_by_category)

# supporting visualization
plt.figure(figsize=(8,5))
sns.barplot(x=revenue_by_region.index, y=revenue_by_region.values)
plt.title("Total Revenue by Region (Cleaned Data)")
plt.xlabel("Region")
plt.ylabel("Total Revenue (₹)")
plt.tight_layout()
plt.show()
```

Output Screenshot:

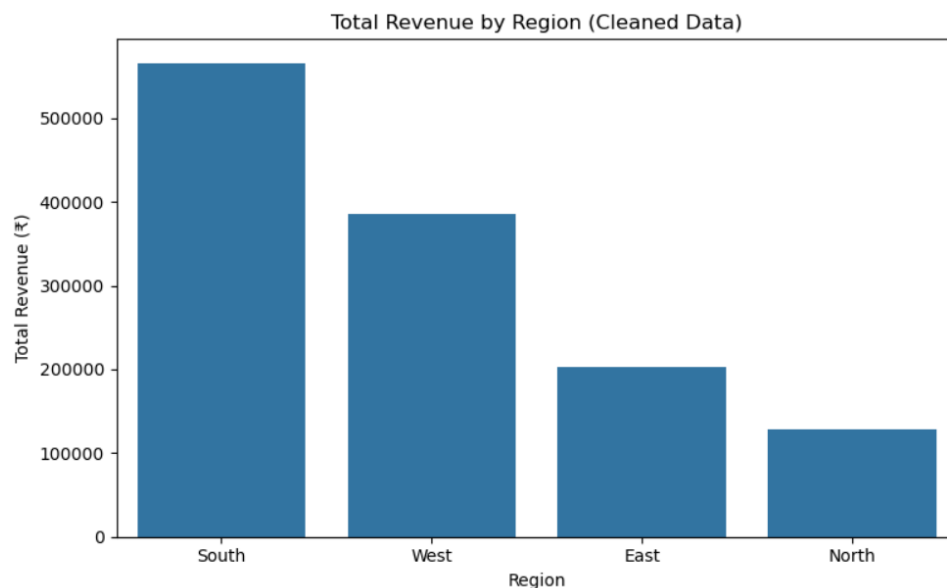
```
Correlation matrix:
            units_sold  unit_price  customer_age  customer_rating \
units_sold      1.000000   -0.000671      0.042585      0.008520
unit_price     -0.000671    1.000000      0.162361     -0.396142
customer_age      0.042585    0.162361      1.000000     -0.320092
customer_rating  0.008520   -0.396142     -0.320092      1.000000
revenue          0.960296    0.239685      0.093548     -0.088483

            revenue
units_sold      0.960296
unit_price      0.239685
customer_age     0.093548
customer_rating -0.088483
revenue         1.000000
Revenue by category:
product_category
Books           493781.60
Home & Kitchen  271066.75
Sports          247681.07
Clothing        185655.31
Electronics     83582.75
```

```

Name: revenue, dtype: float64
Revenue by region:
  region
South   565977.63
West    385363.15
East    202871.51
North   127555.19
Name: revenue, dtype: float64
Payment method share (%):
  payment_method
Credit Card      36.0
Cash on Delivery  30.0
Debit Card       20.0
UPI              14.0
Name: proportion, dtype: float64
Average rating by category:
  product_category
Sports            2.663636
Home & Kitchen   2.933333
Electronics      3.012500
Books            3.175000
Clothing         3.480000
Name: customer_rating, dtype: float64

```



Insights:

- units_sold and revenue are strongly correlated (0.96)** — revenue is driven almost entirely by order volume rather than unit price (`unit_price` vs `revenue` correlation is a weak 0.24), meaning bulk orders matter more to revenue than premium pricing.
- South region dominates revenue** (₹565,978 — 39% of total across 4 regions), nearly 3x North's contribution (₹127,555), showing heavy geographic revenue concentration.
- Sports has the lowest average customer rating (2.66)** while Clothing has the highest (3.48) — despite Sports being a solid revenue contributor, satisfaction there lags noticeably behind other categories, and `unit_price` vs `customer_rating` shows a moderate negative correlation (-0.40), suggesting pricier items tend to get rated lower.

Task 9: Business Recommendations

Explanation: Translates the insights above into 2 actionable, data-backed recommendations.

Recommendation 1 — Investigate and address Sports category satisfaction

Sports has the lowest average rating (2.66) despite being the 3rd highest revenue category (₹247,681). Combined with the negative price-rating correlation (-0.40), this suggests customers may feel Sports items are overpriced relative to quality/value. **Action:** review Sports pricing and quality-control feedback specifically, rather than treating all categories uniformly.

Recommendation 2 — Concentrate marketing spend and inventory on South and West regions

South (₹565,978) and West (₹385,363) together generate ~66% of total revenue across the 4 regions, while North lags far behind (₹127,555). **Action:** prioritize stock availability, regional promotions, and delivery capacity in South/West where demand is proven, while running a smaller-scale test campaign in North to diagnose whether the gap is demand-driven or a marketing/reach problem before committing further budget there.